

Architectural Styles

1. What Is an Architectural Style?

An **architectural style** describes a category of software systems and defines four things:

- 1. **Components** — the major processing units of the system (databases, computational modules, servers, UI modules)
- 2. **Connectors** — how components communicate, coordinate, and cooperate (method calls, message passing, pipes, network protocols)
- 3. **Constraints** — rules restricting how components can be combined or integrated
- 4. **Semantic models** — help designers reason about the overall behavior of the system by examining the known behavior of its individual components

An architectural style is applied to the **entire system**, shaping its overall organization and structure — not one localized piece of it.

Architectural Style vs. Architectural Pattern

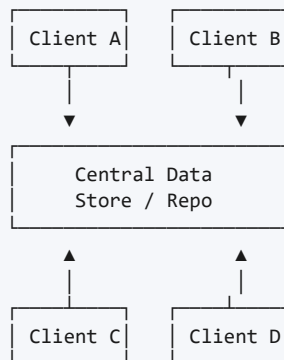
This distinction is a common source of confusion, and a common interview question in its own right:

	Architectural Style	Architectural Pattern (Design Pattern)
Scope	Entire system	One specific aspect of the architecture
Guidance	Shapes the whole system's organization	Rule-based guidance for one infrastructure-level concern (concurrency, communication, resource management)
Orientation	Structural (how everything fits together)	Often behavior-oriented (synchronization in real-time systems, handling interrupts, managing concurrent execution)

(This is the same style-vs-pattern distinction from the Design Concepts topic, restated with architecture-specific examples — a design pattern like Observer is local and behavioral; MVC or Layered, below, are whole-system styles.)

2. Data-Centered Architecture

A central **data store** (a database or file system) sits at the core of the system. Other components — **clients** — access this shared repository to add, modify, delete, or retrieve data.



- The data store may be **passive** — clients access data independently, on their own initiative.
- A variation is the **blackboard architecture**, where the repository actively **notifies** clients when relevant data changes, rather than clients having to poll it.

Advantages: promotes integrability — new clients can be added or modified without affecting existing ones, since they all interact only through the shared repository; supports coordination among clients purely via shared state.

Example: database-centric business applications.

3. Data-Flow Architecture (Pipe-and-Filter)

Used when input data must be transformed into output data through a **series of processing steps**. Each processing unit is a **filter**; filters are connected by **pipes**, which transmit data from one filter to the next.

```
Input → [Filter 1] → pipe → [Filter 2] → pipe → [Filter 3] → Output
```

Key characteristics:

- Each filter operates **independently**
- Filters don't need to know anything about the internal workings of other filters — only the data format flowing through the pipe
- Data flows **sequentially** from one filter to the next

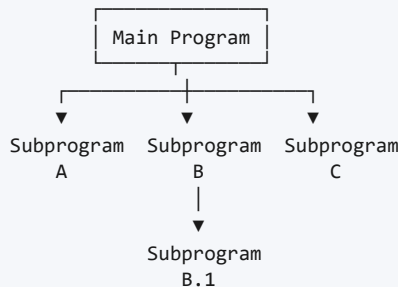
Advantages: easy to understand and modify (each filter is a self-contained unit); high reusability of filters (the same filter can be reused in a different pipeline).

Example: compilers (lexer → parser → optimizer → code generator, each stage a filter), signal-processing pipelines.

4. Call-and-Return Architecture

Organizes the system as a **control hierarchy**, where control flows from a main component down to subordinate components.

a) Main Program / Subprogram Architecture



A main program controls execution; subprograms perform specific tasks and may themselves call further subprograms.

Advantages: simple and easy to implement; well-structured, predictable control flow.

b) Remote Procedure Call (RPC) Architecture

Extends the main/subprogram style **across networked systems** — components are distributed across multiple machines, but a remote call is made to *look and behave* like an ordinary local procedure call from the caller's point of view.

Example: distributed client-server applications.

5. Object-Oriented Architecture

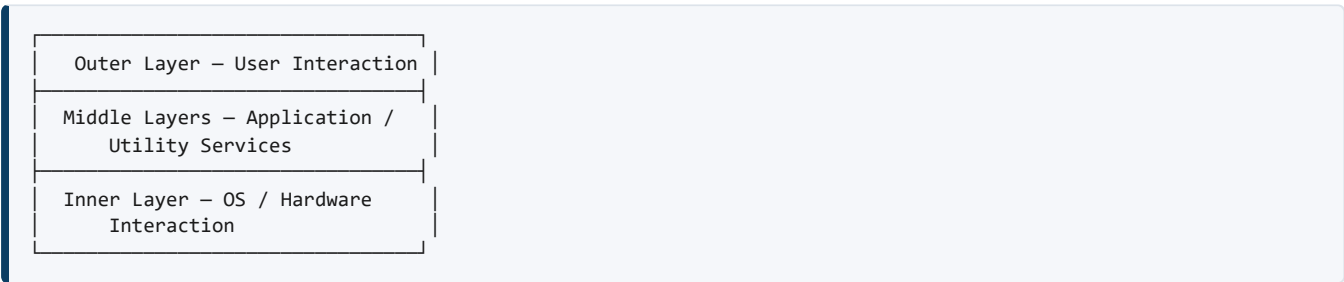
The system is organized as a collection of **objects**, each encapsulating its own data and the operations that manipulate that data. Objects communicate exclusively through **message passing** (i.e., calling each other's methods) rather than reaching into each other's internal state.

Key characteristics: high cohesion *within* each object (its data and behavior are tightly related); loose coupling *between* objects (they interact only through defined interfaces); supports reuse and maintainability — this is precisely the coupling/cohesion "golden rule" from the Design Concepts topic, applied at the whole-system architectural level rather than just the module level.

Example: object-oriented enterprise applications.

6. Layered Architecture

Organizes the system into **hierarchical layers**, each with a specific, well-defined responsibility. A layer can only call the layer directly beneath it (and is called by the layer directly above it).



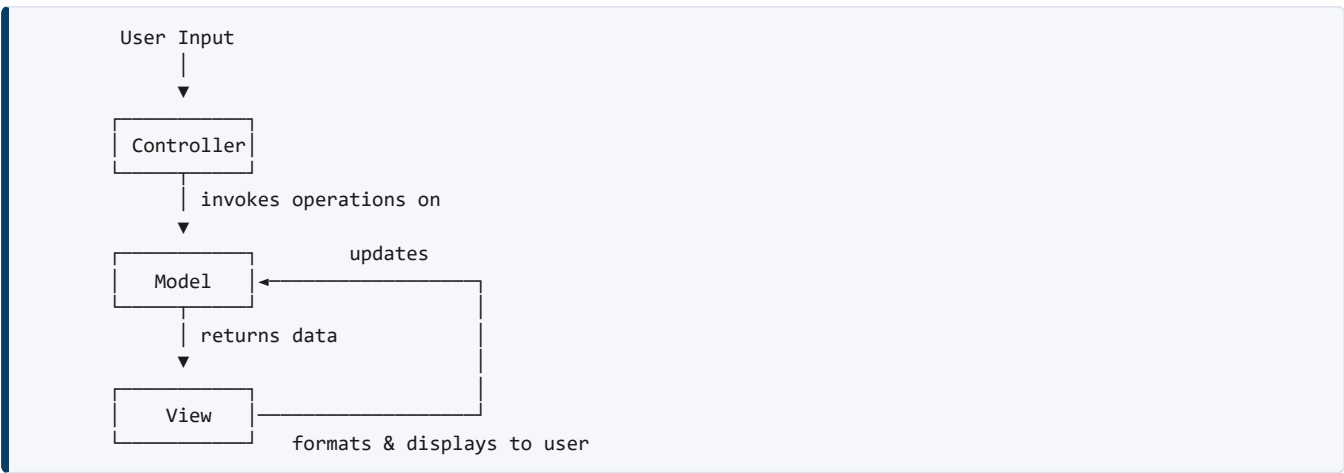
Advantages: clear separation of concerns; easier testing and maintenance (each layer can be tested/replaced somewhat independently); changes in one layer have minimal impact on the others, *as long as* the interface between layers stays stable.

Example: operating systems, network protocol stacks (this is literally how the OSI/TCP-IP model — covered in the CN materials — is structured: each layer only interacts with the ones directly above/below it).

7. Model–View–Controller (MVC) Architecture

A specialized architectural style, extremely common in web and mobile applications, that divides the system into three components:

Component	Responsibility
Model	Application data and business logic
View	Presentation and user interface
Controller	Handles user input and coordinates interaction between Model and View



Working: a user request is handled by the Controller → the Controller invokes operations on the Model → the Model updates its data and returns results → the View formats and displays that data to the user.

Advantages: clear separation of concerns (data logic, presentation, and input-handling never mix); supports parallel development (frontend and backend developers can work against the Model/View boundary independently); improves maintainability.

Interview soundbite: "MVC is really Layered Architecture specialized for interactive applications — the Controller and View together form the 'outer layer' handling user interaction, and the Model is the inner layer holding business logic and data, with the same benefit: changes to how data is presented (View) shouldn't require touching how it's computed (Model), and vice versa."

8. Architectural Archetypes (bridging Requirements → Architecture)

An **architectural archetype** is an abstraction — similar in spirit to a class — that represents a core, recurring element of system behavior. Archetypes:

- Capture the **stable and fundamental** concepts of a system
- Form the initial basis of the system's architecture
- Deliberately contain **no implementation details** yet

A small set of archetypes is usually sufficient even for a complex system, and archetypes are typically **derived from the analysis classes** identified earlier, during requirements analysis (see the Requirements Engineering topic). As design progresses, each archetype is refined into a concrete software component:

Context → Archetypes → Components → Detailed Design

(Your source notes reference a specific hand-drawn diagram illustrating this progression that wasn't extractable as text from the scanned page — the general concept above, from Pressman's treatment of architectural design, is what that diagram illustrates.)

Quick Comparison

Style	Core Idea	Best Fit
Data-Centered	Shared central repository	Database-driven business apps
Data-Flow (Pipe-and-Filter)	Sequential transformation stages	Compilers, signal processing
Call-and-Return	Hierarchical control flow	Traditional structured programs, distributed RPC systems
Object-Oriented	Encapsulated objects, message passing	General-purpose OO enterprise systems
Layered	Hierarchical responsibility layers	OS, network protocol stacks
MVC	Separate data / presentation / control	Web and mobile applications

Interview Questions With Answers

Q1. What four things does an architectural style define, and why does "constraints" need to be one of them separately from "components" and "connectors"?

Answer: An architectural style defines components (the major processing units), connectors (how those components communicate), constraints (rules restricting how components can be combined), and semantic models (which help reason about overall system behavior from the known behavior of individual components). Constraints need to be separate from components and connectors because simply having a set of components and a set of allowed connector types doesn't by itself prevent them from being wired together in ways that violate the style's intent — for example, a Layered architecture's components and connectors alone don't stop someone from wiring the innermost layer directly to the outermost one; it's the *constraint* ("a layer may only call the layer directly beneath it") that actually enforces the hierarchical structure the style is meant to guarantee, so without explicit constraints, a style would only be a vocabulary of building blocks rather than an actual rule for how a system must be organized.

Q2. Explain the difference between the passive-repository variant and the blackboard variant of Data-Centered architecture.

Answer: In the passive-repository variant, the central data store simply holds data and responds to requests — clients access it independently, on their own schedule, to add, modify, delete, or retrieve information, with no initiative coming from the repository itself. In the blackboard variant, the repository is active rather than passive: it actively notifies clients when relevant data changes, rather than requiring clients to repeatedly check ("poll") for updates themselves. The practical difference matters for responsiveness and efficiency — a system where multiple components need to react quickly

to changes made by other components (the classic "blackboard" use case is collaborative problem-solving, where several independent specialist components all contribute partial solutions to a shared space) benefits from the notification-driven blackboard variant, while a simpler system where clients only need data on their own schedule is well served by the simpler passive variant.

Q3. Why is Pipe-and-Filter architecture particularly well suited to compilers, and what specific property of filters makes them reusable?

Answer: A compiler's job naturally decomposes into a strict sequence of transformations on the same underlying artifact — source text becomes a token stream, the token stream becomes a parse tree, the parse tree becomes an optimized intermediate representation, and that becomes generated code — which maps directly onto Pipe-and-Filter's model of data flowing sequentially through independent processing stages. The property that makes filters reusable is that each filter operates independently and doesn't need to know anything about the internal workings of any other filter — it only needs to agree on the data format coming in and going out through its pipes. Because of that isolation, a filter written for one pipeline (say, a specific optimization pass) can be dropped into a different pipeline that produces and expects the same data format, without needing any changes, since the filter was never coupled to the specifics of whatever filter happened to run before or after it in its original pipeline.

Q4. How does Remote Procedure Call (RPC) architecture relate to the Main Program/Subprogram style, and what problem does it solve that the basic style can't?

Answer: RPC architecture is a direct extension of the Main Program/Subprogram style, keeping the same underlying idea — a controlling caller invokes a subordinate unit of work and gets a result back — but applying it across a network rather than within a single process on a single machine. The problem it solves is that the basic Main Program/Subprogram style assumes all subprograms are locally available in the same address space, which breaks down completely once a system needs to be distributed across multiple machines (for scalability, for separating concerns onto different servers, or because different parts of the system are owned by different teams). RPC solves this by making a remote call *appear* to the calling code exactly like an ordinary local procedure call — the caller doesn't have to manually manage network communication, serialization, or remote addressing at the call site — which lets a system retain the simple, hierarchical control flow of Main Program/Subprogram architecture even though the actual subprograms now live on different physical machines.

Q5. In what sense is Object-Oriented architecture just "coupling and cohesion applied at the system level" rather than a fundamentally different concept?

Answer: The defining characteristics of Object-Oriented architecture — high cohesion within each object (its data and the operations on that data are bundled together because they're all related to the same responsibility) and loose coupling between objects (they only interact through message passing/method calls, never by directly manipulating each other's internal state) — are exactly the same two properties, cohesion and coupling, that the Design Concepts topic identifies as the "golden rule" of good module-level design. The difference is purely one of scale: at the module level, the units being evaluated for cohesion and coupling are individual functions or modules; at the object-oriented

architectural level, the units are entire objects (each object being, in effect, a module that bundles data with the operations on that data). This is a good illustration of a broader theme — many "different" design and architecture concepts are really the same small set of underlying principles (cohesion, coupling, separation of concerns) reapplied at different levels of granularity, from a single function up to the architecture of an entire system.

Q6. Explain the control flow of a single user request in MVC, step by step, and identify which component would need to change if the business rule for calculating a value changed versus if just the visual layout changed.

Answer: A user request is first received by the Controller, which interprets the request and invokes the appropriate operation on the Model; the Model then updates its internal data according to that operation and returns the result; finally, the View takes that result and formats it for display back to the user. If a business rule changed — for example, how a total price is calculated — only the Model needs to change, since business logic lives there exclusively; the Controller and View remain untouched because neither of them contains business logic, only request-handling and presentation logic respectively. If instead just the visual layout changed — say, displaying the same price in a different currency format or a different part of the screen — only the View needs to change, since the underlying data and the logic that produced it in the Model haven't changed at all, only how that data is presented. This clean separation, where a given kind of change maps to exactly one component, is the concrete payoff of MVC's separation of concerns.

Q7. What is an architectural archetype, and how does it relate to the analysis classes covered in Requirements Engineering?

Answer: An architectural archetype is an abstraction, similar in spirit to a class, that captures a core, stable, and fundamental element of a system's behavior — deliberately without any implementation detail yet, since it exists specifically to represent the *concept*, not the eventual code. Archetypes are typically derived directly from the analysis classes that were identified during requirements engineering's elaboration task (the business entities identified while refining raw requirements into a structured model) — the relationship being that analysis classes represent the problem domain's core entities from a requirements perspective, and architectural archetypes take those same core, stable entities and use them as the starting skeleton for the system's architecture, before that skeleton is progressively refined (per the Context → Archetypes → Components → Detailed Design progression) into concrete, implementable software components.

Q8. Scenario: A team is building a compiler-like tool that reads a config file, validates it, transforms it into an internal format, and then generates several different output file formats from that internal representation. A junior engineer suggests using MVC because "it's the standard architecture." Is that the right call, and which style would actually fit better?

Answer: MVC is not the right call here — MVC's entire value proposition is separating data, presentation, and user-input-handling for an *interactive* system where a user is actively viewing and

manipulating data through a UI, and this tool has none of those elements: there's no user interface being presented, no user input being handled interactively, and no "view" of anything in the MVC sense. What this tool actually is, is a sequence of transformations applied to data — read, validate, transform, generate multiple outputs — which is the textbook use case for Data-Flow (Pipe-and-Filter) architecture: each stage (reading, validating, transforming, each output generator) can be implemented as an independent filter connected by pipes, each filter needing to know nothing about the others' internals, and new output formats could be added later simply by adding a new filter at the end of the pipeline without touching the earlier stages at all. This is a good illustration of why "use the standard/popular architecture" is the wrong instinct — the right architectural style should match the actual shape of the problem (a linear data transformation pipeline, here), not whichever style happens to be most commonly discussed.